Reference Data Service

- Basic Info
- Background
  - So what are AssetLocations?
  - So what does Reference Data Service provide
  - Who are the users of the service?
- Code
- Database
- Web Servers
  - Read Nodes vs Write Node
  - Server Details
    - Swagger
    - Virtual Machines
- Authentication
- GUI
- Automatic Syncing of RDS
- How can I use RDS from Rapid Engine?
- Releases
  - Deploying the Backend
  - Deploying the GUI
- Support and Troubleshooting
  - Useful commands to run on the machine
  - Load balancer Issues

Basic Info

- Database details:
  - url: db1-refsvc.g20api.com : 5432
  - Login details: Please see LastPass
- UI Table Works:
  - https://refsvc.prod.g20api.com:4321/sign-in
  - (alt link): https://refsvc.g20api.com:4321/
  - Login: as per 'old' dashboatd without SSO, chat to @Yogesh Sharma
- UI Code: **Unavailable**, properly of Palinorio, https://www.palinor.io/, @Haris Lazanitis company
- API Access / Swagger Docs:
  - Write endpoints: https://refsvc.prod.g20api.com/ref-write-svc/docs
  - Read endpoints: https://refsvc.prod.g20api.com/ref-svc/docs
  - Use user: test, password: test, for read only user

    - Ask @Yogesh Sharma to give you write user if you need to
- API Code: https://github.com/G20Strategies/rapid-engine/tree/master/projects/refsvc
- Runs on: ap-tk04-refsvc box

Background

Reference Data Service is meant to be the central reference of G20 internal reference data. As of now (Jan 2025) this only applies to Asset Locations, but the plan is for this to evolve to become a centralized reference for everything including market data, symbols, trading accounts, portfolios etc.

With that in mind, for the rest of this document when talking about “Reference Data Service” we just mean Asset Locations.

So what are AssetLocations?

Have a look at Reference Database Schema for earlier dicussion of this. AssetLocation, is an internal representation of any real-world structure that holds assets. This includes:

- Exchange Accounts / Subaccounts
- Fireblocks Accounts
- Any other custodian accounts.
- AMM pools
- Crypto Wallets
- Bank Accounts
- Broker Accounts

As of now, we only modeling the first two.

So what does Reference Data Service provide

Reference Data Service provides:

- A database of asset locations
- A workflow / gui to manage asset locations (create, update, tag, set balance thresholds etc.)
- A Web API for services requiring access to Asset Location data.

Once the data set is completed we need be in a situation where:

- Any exchange subaccount is mapped and registered in RDS
- Any Fireblocks Location is mapped registered
- etc. etc.

This means that we can have an internal representation of our risk driven off Asset Location that everything can be built around.

Who are the users of the service?

As a direction, anything refers to an asset location should be driven off the RDS. Examples are:

- Balances
- Trades
- Orders
- Perp Positions
- Rebalancing
- Risk Reports
- Treasury App: Treasury reserves location ids from 100 to 999 for its own development. Information about those location ids can be found from the Treasury API (FastAPI - Swagger UI)

Code

The code is in the same repository as rapid engine as a separate project:
https://github.com/G20Strategies/rapid-engine/tree/c82ef00d76ca4f8274b1885f32494b6b34b2b4b0/projects/refsvc Restricted content

Database

Note Applications should not query the database directly, they should query through the API

The database is in RDS in Tokyo.

`reference-database-tokyo.cvxpra6z19sf.ap-northeast-1.rds.amazonaws.com:5432/Reference_database`

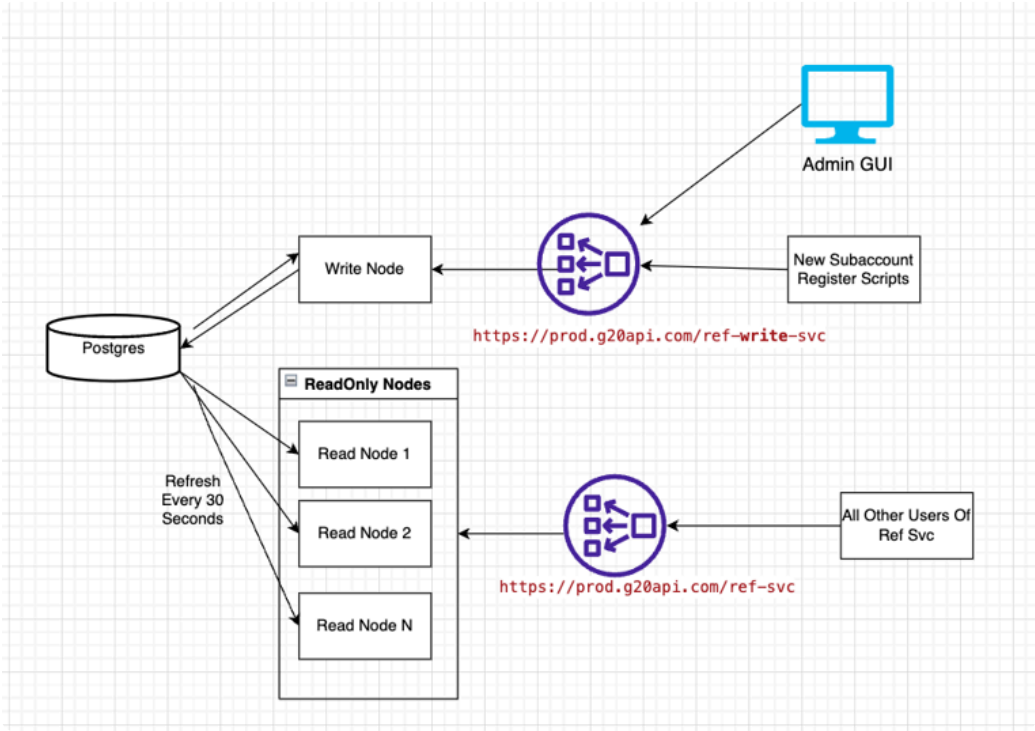
Table Schema:

https://github.com/G20Strategies/rapid-engine/tree/c82ef00d76ca4f8274b1885f32494b6b34b2b4b0/projects/refsvc/database/ddl/postgresql Restricted content

Note, the data is fully auditable, meaning you can refer to the state of the data at any point of time in history. This is using uni-temporal chaining W Temporal database and all updates are fully auditable.

Web Servers

The application is exposed with REST. The current design is built with high availability in mind, assuming a heavy read workflow and a moderate write workflow. Hence we have a single write server and multiple read servers.



Read Nodes vs Write Node

Read Nodes and write Nodes are identical, except that it's instantiated with the `NODE_TYPE=WRITE` parameter while read nodes are instaniated with `NODE_TYPE=READ` parameter.

This ensures two things:

(1) Write node is NOT accessible to users that don't have write access. If you don't have write access, you cannot even do read operations on the read node:

```
async def get_user_from_token(
    security_scopes: SecurityScopes,
    token: Annotated[str, Depends(oauth2_scheme)],
    context: AppContext =  Depends(get_app_context),

):
    current_user = await context.token_checker(security_scopes, token)
    if context.node_type == NodeType.WRITE:
```

```
        # Even for R/O operations access to write nodes would requre write operations.
        validate_user_has_access_to_scope(current_user, 'write', is_websocket=False)
    return current_user
```

(2) READ nodes, even though they still expose the WRITE endpoints, they don't allow it to be called by any of the users.

```
async def get_write_user_from_token(
    current_user = Depends(get_user_from_token),
    context: AppContext =  Depends(get_app_context),

):
    if context.node_type == NodeType.READ:
        raise_unauthorized_exception("This is a READ only node. Update operations not allowed", is_websocket=False)
    validate_user_has_access_to_scope(current_user, 'write', is_websocket=False)
    return current_user

```

Server Details

The nodes are accessed from EC2 **Load Balancers**, and all the endpoints are **authenticated** and **whitelisted** behind **SSL.**

Swagger

https://refsvc.prod.g20api.com/ref-write-svc/docs

https://refsvc.prod.g20api.com/ref-svc/docs

Virtual Machines

As of now we have two READ and one write node. All of them are currently deployed on the same machine (not ideal, will need to change later for better redundancy).

They are deployed as docker containers. The docker images are stored in our AWS container registry

The machine public IP is:

35.75.19.180

ssh details should be in LastPass

Authentication

The write node also acts as a authentication server, issuing JWT tokens using username and password.
We are using RS256, asymmetric hashing. The write node is instantiated with access to the private key, while all other ndoes only have access to the public key. So all nodes can confirm a valid JWT but only the read node can issue new ones.

For convinience the tokens are currently issued with long expiry so they can be used as API keys. (will need to look into how to mitigate that)

Note the idea is that all other apps will be authenticated using the same keys.

The user names and passwords are currently managed on the app-side. We store the username and the password hashes on the local machine (we don't store the password).

Asof now the users are manually created. I created the users with the same credentials as what what is used in the dashboard, but the two are not automaticaly synced. All users are created with both read and write access.

We additonally have 2 system users:

- **prod_system_reader**, with read access (pwd in lastpass). This should be the main user for almost all systematic access to the service (we should probably create more system read users, one per app).
- **prod_system_writer**, with RW access. This should only be used for processes that manuallly sync the RDS from exchanges etc.

GUI

GUI is behind the VPN and is authenticated with username and password. (see note on authentication)

It should be used primarily by operations to manage the data.

https://refsvc.g20api.com:4321/

Its deployed on the same VM as the back-end servers (see above) as a container.

Note the the GUI is an instance of third party tool that TableWorks, which is a third-party solution that G20 got (for free) from Palinorio Technologies.

The tool is built on the idea that if you builld the back-end with proper opeanapi documentation (which we do) it infurs your CRUD operations and creates an admin GUI from you without any front-end setup. You can additionally pass more configuration changes using a special gui endpoint which we do here: https://github.com/G20Strategies/rapid-engine/blob/haris-refsvc-connectors/projects/refsvc/refsvc/app/routers/gui.py#L167

>> Pending >> Add some user-manual for how to use the GUI for this workflow <<

Automatic Syncing of RDS

As of now RDS is synced up with fireblocks and exchanges using Rapid Engine Connectors.

This is ran manually by Haris atm (shoudl be automated soon), but it essetially using `connector_refsvc_bridge_utils`.`sync_refdb_with_subaccounts_from_exchange`

(I will put more stuff on this once it gets productionized)

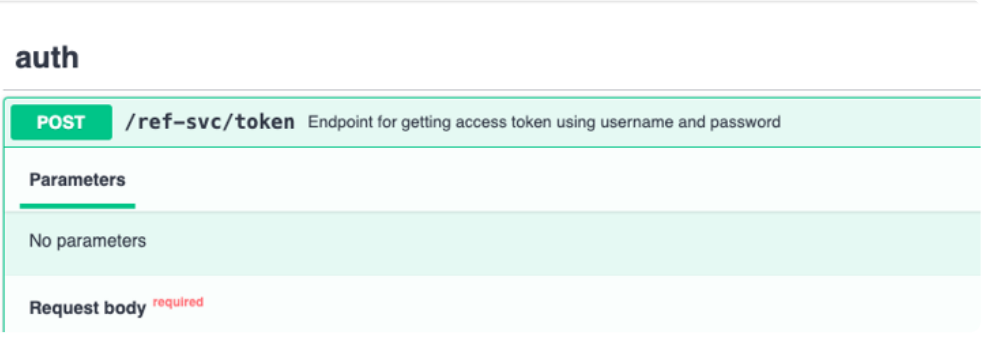
How can I use RDS from Rapid Engine?

We created some adaptors (client code) in rapid engine for developers to be able to start writing processes that require access to refsvc. You can of course always use Swagger (Reference Data Service | Swagger) to explore the API and write your own code in any language, but anything that comes from rapid is easier to use the adaptor code for now (we can write client libraries later)

Because this is an authenticated API you will need to pass the JWT.

So, step-by-step, to access the service through the adaptor you should:

1. Store the JWT token in `SECRETS_REFERENCE_SERVICE_TOKEN` variable
   a. Go to swagger for the write node (it has to be the write node) and create a token with your username and password (or if you are preparing to release, you can use the system user in Lastpass (see above))

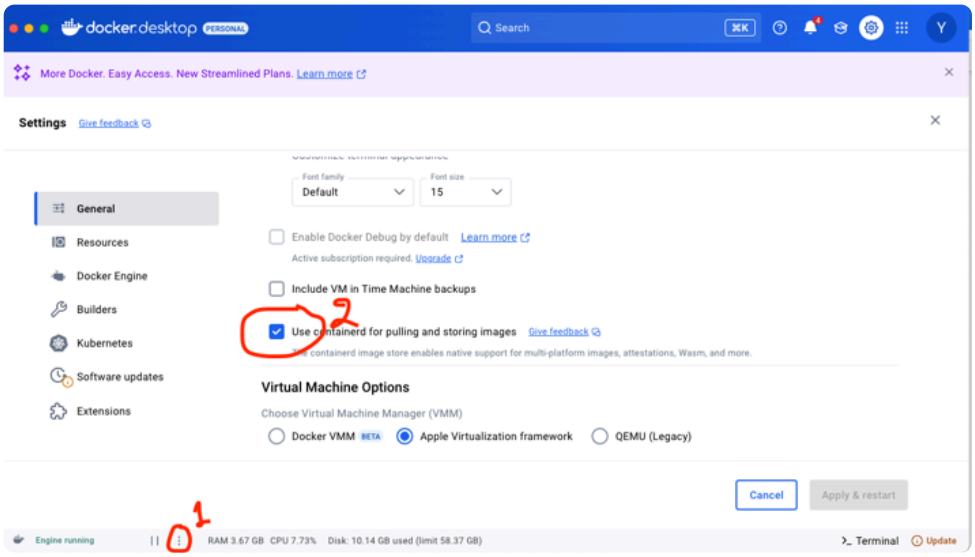


   b. Copy the JWT token
   c. Paste the token on your .env file or however you register reference environement variables
2. Explore and run functions in `connector_refsvc_bridge_utils.py`. If what you need is not there extend the library.

Releases

Deploying the Backend

1. Run `./build_containers.sh --push`
   a. You might need to enable 'containerd' to be able to build multi-arch images.



2. Login to refsvc server: 35.75.19.180 [`ap-tk04-refsvc` machine, see AWS Machine Registry]
3. `cd /home/ubuntu/appdata/refsvc`
4. `./docker-run.sh <VERSION>`
5. Update database schema
   a. Generate ddls `./database/utilities/create_ddl_from_sqlalchemy.py`
   b. Login to DB and execute manually
   c. TODO: Should we have migration process instead? Alembic? @Dmitry Rusakov - FYI

Deploying the GUI

... add some info ...

Support and Troubleshooting

Useful commands to run on the machine

```
#### View containers status
docker ps -a

#Note: refsvc_8021, refsvc_8000 -> Read Nodes
#      refsvc_8099 -> Write Node
#      gui_4321 -> GUI
```

```
7
8  ### Tail logs
9  docker logs -f --tail 100  refsvc_8000
10
11 #### Stop Container
12 docker stop refsvc_8021
13
14 ### Restart container
15 docker restart refsvc_8099
16
17 ### Check the health of severs
18 curl http://localhost:8099/ref-write-svc
19 curl http://localhost:8000/ref-svc
20 curl http://localhost:8021/ref-svc
21
22
```

Load balancer Issues

Though the load balancer that sits in front of services should be an AWS-managed resilient service, any issues with the service have a possibility of being related to the load balancer (e.g. in cases of breaking configurations).

If you are able to log into the machine and rule out issues with the services themselves, for load balancer issues someone with AWS access will need to look inot it (atm. it should be Jackson, Yogesh, Dmitiry or Haris)

The LB should in in [Tokyo] > EC2 > Load balancers > Reference-service-lb